SIH 2026 PROTOTYPE MANUAL

SAHAYAK — FULL SYSTEM ARCHITECTURE & TECH STACK

A Comprehensive Developer & Teammate Guide for Smart India Hackathon (SIH 2026)

Purpose: This document is designed for the entire Sahayak engineering team (lead, frontend developers, backend developers, and flowtesters). It translates the code we built into plain, straightforward technical explanations so that any team member can confidently explain every layer of our platform during presentations and viva questions.

1 The Big Picture: Three-Tier Cooperative Platform

Overview: Sahayak connects urban household customers with verified labour cooperative workers (electricians, plumbers, cleaners, technicians) without corporate exploitation. It operates on a 3-tier architecture:

Layer	Technology	Deployment / Host	Role in Sahayak
1. Mobile Apps	Flutter 3.x / Dart	Local Web / APK	Dual Customer & Worker Mobile Portals
2. Admin Web	React 18 + TS + Vite	Vercel Cloud	Cooperative Federation Dispatch Command
3. Cloud Backend	FastAPI (Python 3.11)	Render Cloud	High-Speed Asynchronous REST Engine
4. Database	PostgreSQL 15	Supabase Cloud	10 Tables, Auth Engine & Role Checking

The 30-Second Elevator Pitch (Memorize This!)

"Sahayak is a three-sided, cooperative-owned digital gig marketplace that replaces corporate aggregators. Customers discover federation-verified home services via Flutter mobile; workers manage jobs and availability on their portal; and the Cooperative Federation monitors live dispatches and worker welfare via a real-time React admin dashboard."

2 Mobile Layer: Flutter & Dual-Role Dynamic Routing

How it works: We built both the Customer and Worker applications using a single, unified Flutter codebase. The app uses an intelligent startup router called AuthGate.

1. When the app launches, AuthGate checks device storage (SharedPreferences) for an existing session token (auth_token).
2. If role == 'CUSTOMER', the app immediately opens CustomerHomeScreen (catalog search, category chips, service details, and book now).
3. If role == 'WORKER', the app opens WorkerHomeScreen (real-time on-duty switch, weekly availability calendar, and incoming jobs feed).
4. If no session is found, AuthScreen is displayed, allowing user registration with role selection (Customer vs Worker).

3 Cloud Backend Layer: FastAPI & Production REST APIs

Backend Engine: Our backend is written in FastAPI (Python) and hosted live on Render. It enforces strict type contracts using Pydantic v2 and connects to Supabase using a service role client.

Route Group	Endpoint	Method	Action Performed
Auth	/api/v1/auth/register	POST	Creates Supabase Auth user & profile (Customer/Worker)
Auth	/api/v1/auth/login	POST	Authenticates password hash & returns role-based JWT
Services	/api/v1/services	GET	Queries live catalog from Supabase with category filter
Bookings	/api/v1/bookings	POST	Inserts customer order with status='PENDING'
Bookings	/api/v1/bookings	GET	Fetches live bookings joined with customer profile
Bookings	/api/v1/bookings/{id}/status	PATCH	Updates status: ACCEPTED -> IN_PROGRESS -> COMPLETED
Admin	/api/v1/admin/workers	GET	Fetches registered workers for federation review
Admin	/api/v1/admin/workers/{id}/verify	PATCH	One-click worker verification toggle (VERIFIED/REJECTED)

Why Did We Use Integer Paise for Prices? (Crucial Financial Design)

In Supabase and FastAPI, all prices are stored in paise (e.g. 49900 = ₹499.00).

Why? In programming, standard decimal floats suffer from IEEE 754 precision rounding errors (e.g. 0.1 + 0.2 = 0.30000000000000004).

Storing integer paise guarantees zero rounding discrepancy in GST deductions and worker dividend splits!

4 Database Architecture: 10 Relational Supabase Tables

Schema Design: Our PostgreSQL database on Supabase organizes the entire cooperative lifecycle across 10 specialized tables:

Table Name	Primary Key	Core Columns & Types	Business Purpose
profiles	user_id (UUID)	full_name, phone, role (ENUM)	Master user identity & role tag
worker_profiles	worker_id (UUID)	skills (TEXT[]), rating_avg, verification	Tradesperson badges & status
services	service_id (UUID)	name, category, base_price_amount (INT)	Federation approved rate card
bookings	booking_id (UUID)	service_id, customer_id, worker_id, status	Live gig lifecycle tracker
worker_availability	availability_id (UUID)	day_of_week, start_time, end_time	Weekly on-duty schedule
payments	payment_id (UUID)	booking_id, amount, payment_status	Digital settlement records
reviews	review_id (UUID)	booking_id, rating (1-5), comment	Customer quality audit
complaints	complaint_id (UUID)	raised_by, description, status	Cooperative grievance redressal
worker_welfare	welfare_id (UUID)	worker_id, fund_type, amount_accrued	e-Shram / pension contributions
notifications	notification_id (UUID)	user_id, title, message, is_read	Real-time dispatch alerts

5 Cooperative Federation Admin Web: React 18 & Vite

Federation Portal: The admin portal in admin-web/ is built with React 18, TypeScript, Tailwind CSS, and Vite, deployed globally on Vercel.

- Real-Time Polling Loop: Runs an asynchronous 8-second polling cycle that fetches /bookings and /workers, re-rendering the live dispatch monitor and recalculating KPI tiles automatically without manual browser refresh.
- Interactive Booking Inspection Modal: Clicking any booking row opens a customer dispatch inspector showing notes, phone, map coordinates, and manual status override buttons.
- Instant Optimistic Verification: When federation officers click 'Approve', the UI immediately reflects the verified badge while synchronizing with the database in the background.

6 The 'Hero Demonstration Loop' Walkthrough (Live Demo Steps)

Live Demo Script: Follow these exact steps during the live jury demonstration across your devices:

Step	Device	User Action	Visible Result on Screens
1	Laptop (Admin)	Open Vercel Dashboard	Live KPI cards show active bookings and verified workers.
2	Phone 1 (Customer)	Book 'Home Deep Cleaning'	Green toast appears; order is saved as 'PENDING' in database.
3	Laptop (Admin)	Watch Dispatch Table	New booking row pops up with Orange [Pending Dispatch] badge.
4	Phone 2 (Worker)	Tap 'Accept' in Incoming Jobs	Job turns ACCEPTED; assigned worker is locked in cloud DB.
5	Laptop (Admin)	Watch Status Badge	Row updates to Blue [Worker Accepted] with technician assigned.
6	Phone 2 (Worker)	Tap 'Start' -> 'Complete Job'	Worker earnings increment; status updates to COMPLETED.
7	Laptop (Admin)	Observe Final Settlement	Row turns Green [Completed]; Platform Revenue metric increases!

7 Judge Q&A & Viva Cheat-Sheet (Top 4 Questions)

Q1: How does the mobile app know whether to open Customer or Worker screen? A: "During registration/login, our FastAPI backend checks public.profiles and returns the user's role. Flutter stores this in SharedPreferences. On app launch, AuthGate reads the role and opens either CustomerHomeScreen or WorkerHomeScreen."

Q2: What is the core market advantage over Urban Company? A: "Urban Company takes 25%-30% commissions and treats workers as disposable freelance supply. Sahayak is owned by the Labour Cooperative Federation: platform overhead is only 3%-5%, workers keep 95%+ of their payout, every technician is vetted by state societies, and completed jobs contribute to e-Shram health and pension funds."

Q3: What makes this architecture scalable? A: "Our backend is built in FastAPI (Python) using asynchronous endpoints running on Uvicorn ASGI on Render, and our database is PostgreSQL 15 on Supabase with connection pooling and indexed foreign keys."